```
----------**********----------**********----------
---------**********-----
--- #Order and Sales Analysis#---------
-----Total Order and Total Revenue

SELECT
    COUNT(*) AS total_orders,
    SUM(order_amount) AS total_revenue
FROM customer_orders;


-- Order status distribution
SELECT
    order_status,
    COUNT(*) AS order_count,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 2) AS percentage,
    SUM(order_amount) AS total_amount
FROM customer_orders
GROUP BY order_status
ORDER BY order_count DESC;

---**Almost Equal amount of distribution in order satuts .i.e almost 33% and around  5000 order
count , althogu shipped order cunt little  .i.e 4874**


----- Orders by Status
SELECT
    order_status,
    COUNT(*) AS order_count,
    SUM(order_amount) AS total_sales
FROM customer_orders
GROUP BY order_status;

---- Monthly Revenue Trend

 SELECT
    DATEFROMPARTS(YEAR(order_date), MONTH(order_date), 1) AS order_month,
    COUNT(*) AS monthly_orders,
    SUM(order_amount) AS monthly_revenue
FROM customer_orders
GROUP BY DATEFROMPARTS(YEAR(order_date), MONTH(order_date), 1)
ORDER BY order_month;
```

```
-- 1.2 Monthly sales trend
SELECT
    YEAR(order_date) AS year,
    MONTH(order_date) AS month,
    COUNT(*) AS order_count,
    SUM(order_amount) AS total_sales
FROM customer_orders
GROUP BY YEAR(order_date), MONTH(order_date)
ORDER BY year, month;

--***There is no such change or trend in montly sales , or sale in year , there is almost same
around 210-240 order each mnoth every year**


--- Highest Order Amount
SELECT
    order_id,
    customer_id,
    order_amount
FROM customer_orders
ORDER BY order_amount DESC;

----  Highest Order Amount
SELECT TOP 1 WITH TIES
    order_id,
    customer_id,
    order_amount
FROM customer_orders
ORDER BY order_amount DESC;

--****The  highest order amount is 499.9 ******

--  Average order value by status
SELECT
    order_status,
    AVG(order_amount) AS avg_order_value,
    MIN(order_amount) AS min_order_value,
    MAX(order_amount) AS max_order_value
FROM customer_orders
GROUP BY order_status;
--***Same  average for all order status **
--_____
_____--
-----#Customer Analysis#----------
```

----Total and Unique Customers

```
SELECT COUNT(DISTINCT customer_id) AS total_customers
FROM customer_orders;
```

--**There are almost 7334 customers**

```
--  Customer order frequency
SELECT
    customer_id,
    COUNT(*) AS order_count,
    SUM(order_amount) AS total_spent,
    MIN(order_date) AS first_order_date,
    MAX(order_date) AS last_order_date
FROM customer_orders
GROUP BY customer_id
ORDER BY order_count DESC;
```

----- Repeat Customers (more than 1 order)

```
SELECT
    customer_id,
    COUNT(order_id) AS total_orders
FROM customer_orders
GROUP BY customer_id
HAVING COUNT(order_id) > 1
ORDER BY total_orders DESC;
```

--**** The most no of order repeated is 8*******

```
--  Repeat customers (customers with >1 order)
SELECT
    COUNT(*) AS repeat_customer_count
FROM (
    SELECT customer_id
    FROM customer_orders
    GROUP BY customer_id
    HAVING COUNT(*) > 1
) AS repeat_customers;
```

---******* There are 4402 repeated customer out of 7334 customer , Means good customer retention.***

```
-----Average Orders per Customer
SELECT
   AVG(order_count) AS avg_orders_per_customer
FROM (
   SELECT customer_id, COUNT(order_id) AS order_count
   FROM customer_orders
   GROUP BY customer_id
) AS customer_order_counts;

--*********AVG orders per customer is 2 **************************
--  Time between orders for repeat customers
WITH customer_orders_ranked AS (
   SELECT
      customer_id,
      order_date,
      LEAD(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) AS
next_order_date
   FROM customer_orders
)
SELECT
   customer_id,
   DATEDIFF(day, order_date, next_order_date) AS days_between_orders
FROM customer_orders_ranked
WHERE next_order_date IS NOT NULL;


------------------
WITH customer_orders_ranked AS (
   SELECT
      customer_id,
      order_date,
      LEAD(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) AS
next_order_date
   FROM customer_orders
)
SELECT

   AVG(DATEDIFF(day, order_date, next_order_date) )AS Avg_days_between_orders
FROM customer_orders_ranked
WHERE next_order_date IS NOT NULL;
--------------******Average  time between order repeated is 479***********

-----Monthly Unique Customers
```

```
SELECT
   DATEFROMPARTS(YEAR(order_date), MONTH(order_date), 1) AS order_month,
   COUNT(DISTINCT customer_id) AS unique_customers
FROM customer_orders
GROUP BY DATEFROMPARTS(YEAR(order_date), MONTH(order_date), 1)
ORDER BY  DATEFROMPARTS(YEAR(order_date), MONTH(order_date), 1);

--********Around 220-240 unique customer each month each year **********

-----Customer Segmentation by Order Count

 WITH cust_orders AS (
   SELECT customer_id, COUNT(order_id) AS order_count
   FROM customer_orders
   GROUP BY customer_id
),
segmented AS (
   SELECT
      CASE
         WHEN order_count = 1 THEN 'One-time'
         WHEN order_count BETWEEN 2 AND 3 THEN 'Occasional'
         ELSE 'Frequent'
      END AS segment
   FROM cust_orders
)
SELECT segment, COUNT(*) AS customer_count
FROM segmented
GROUP BY segment;

--*******There is less frequent coustomer .i.e. 816 than there is  one time (2932) and Occasional
customers (3586)*******
--_____
_____--

-----##Payment Status Analysis##----------

----Payment Status Summary

SELECT
   payment_status,
   COUNT(*) AS transaction_count,
   SUM(payment_amount) AS total_amount
FROM payments
```

```
GROUP BY payment_status;

--  Payment status distribution
SELECT
    payment_status,
    COUNT(*) AS payment_count,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 2) AS percentage,
    SUM(payment_amount) AS total_amount
FROM payments
GROUP BY payment_status
ORDER BY payment_count DESC;

--******Around same distribution of payment status (pending failed completed) .i.e 33% or  5000
transactional _count**********

----Payment Method vs Status

SELECT
    payment_method,
    payment_status,
    COUNT(*) AS count,
    SUM(payment_amount) AS amount
FROM payments
GROUP BY payment_method, payment_status
ORDER BY payment_method;



----- Payment Failure Rate by Method

SELECT
    payment_method,
    SUM(CASE WHEN payment_status = 'failed' THEN 1 ELSE 0 END) AS failed_txn,
    COUNT(*) AS total_txn,
    ROUND(100.0 * SUM(CASE WHEN payment_status = 'failed' THEN 1 ELSE 0 END) /
COUNT(*), 2) AS failure_rate_pct
FROM payments
GROUP BY payment_method;

--  Orders with payment issues
SELECT
    co.order_id,
    co.customer_id,
    co.order_date,
```

```
    co.order_amount,
    co.order_status,
    p.payment_status,
    p.payment_amount,
    (co.order_amount - ISNULL(p.payment_amount, 0)) AS amount_due
FROM customer_orders co
LEFT JOIN payments p ON co.order_id = p.order_id
WHERE p.payment_status <> 'completed' OR p.payment_id IS NULL;
--_____________________________________________________________________________
_____________________________________________--

------##Order Details Report#----------------------
----Combined Order-Payment Report

SELECT
    o.order_id,
    o.customer_id,
    o.order_date,
    o.order_amount,
    o.order_status,
    p.payment_id,
    p.payment_date,
    p.payment_amount,
    p.payment_method,
    p.payment_status
FROM customer_orders o
LEFT JOIN payments p ON o.order_id = p.order_id;

---Orders with Missing Payments

SELECT *
FROM customer_orders o
LEFT JOIN payments p ON o.order_id = p.order_id
WHERE p.order_id IS NULL;

--- ***There is around  5500  orders with missing  payments or null value***********8

----Delivered Orders with Failed Payments


SELECT
    o.order_id, o.order_status, p.payment_status, p.payment_method
FROM customer_orders o
JOIN payments p ON o.order_id = p.order_id
```

```
WHERE o.order_status = 'delivered' AND p.payment_status = 'failed';


--- ***There is around  1650 delivered orders with failed payment***********8
-- 4.1 Comprehensive order report
SELECT
    co.order_id,
    co.customer_id,
    co.order_date,
    co.order_amount,
    co.order_status,
    co.shipping_address,
    p.payment_id,
    p.payment_date,
    p.payment_amount,
    p.payment_method,
    p.payment_status,
    CASE
        WHEN p.payment_status = 'completed' AND p.payment_amount >= co.order_amount
THEN 'Fully Paid'
        WHEN p.payment_status = 'completed' AND p.payment_amount < co.order_amount THEN
'Partially Paid'
        WHEN p.payment_status = 'failed' THEN 'Payment Failed'
        WHEN p.payment_id IS NULL THEN 'No Payment Record'
        ELSE 'Payment Pending'
    END AS payment_summary
FROM customer_orders co
LEFT JOIN payments p ON co.order_id = p.order_id
ORDER BY co.order_date DESC;

--_____
_____---

//////////////////////**********************************************************************************************
**********/\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\
--_____---------- Customer Retention
Analysis_____----------
-- Customer retention cohorts
WITH first_orders AS (
    SELECT
        customer_id,
        MIN(order_date) AS first_order_date,
        DATEFROMPARTS(YEAR(MIN(order_date)), MONTH(MIN(order_date)), 1) AS
cohort_month
```

```
    FROM customer_orders
    GROUP BY customer_id
),
monthly_orders AS (
    SELECT
        fo.customer_id,
        fo.cohort_month,
        DATEDIFF(month, fo.cohort_month, DATEFROMPARTS(YEAR(co.order_date),
MONTH(co.order_date), 1)) AS month_number
    FROM first_orders fo
    JOIN customer_orders co ON fo.customer_id = co.customer_id
    GROUP BY
        fo.customer_id,
        fo.cohort_month,
        DATEDIFF(month, fo.cohort_month, DATEFROMPARTS(YEAR(co.order_date),
MONTH(co.order_date), 1))
),
cohort_sizes AS (
    SELECT
        cohort_month,
        COUNT(DISTINCT customer_id) AS cohort_size
    FROM first_orders
    GROUP BY cohort_month
),
retention_counts AS (
    SELECT
        cohort_month,
        month_number,
        COUNT(DISTINCT customer_id) AS retained_customers
    FROM monthly_orders
    GROUP BY cohort_month, month_number
)
SELECT
    FORMAT(rc.cohort_month, 'yyyy-MM') AS cohort,
    rc.month_number,
    cs.cohort_size,
    rc.retained_customers,
    ROUND((rc.retained_customers * 100.0 / cs.cohort_size), 2) AS retention_rate
FROM retention_counts rc
JOIN cohort_sizes cs ON rc.cohort_month = cs.cohort_month
WHERE rc.month_number >= 0
ORDER BY rc.cohort_month, rc.month_number;
----------_____________________________________________________________________
_____________________________________________----------
```

```
-----------------------------------------------------------------------------------------------------------------------------------------
-----------------------------------------
CREATE VIEW  cohort AS
WITH first_orders AS (
    SELECT
        customer_id,
        MIN(order_date) AS first_order_date,
        DATEFROMPARTS(YEAR(MIN(order_date)), MONTH(MIN(order_date)), 1) AS
cohort_month
    FROM customer_orders
    GROUP BY customer_id
),
monthly_orders AS (
    SELECT
        fo.customer_id,
        fo.cohort_month,
        DATEDIFF(month, fo.cohort_month, DATEFROMPARTS(YEAR(co.order_date),
MONTH(co.order_date), 1)) AS month_number
    FROM first_orders fo
    JOIN customer_orders co ON fo.customer_id = co.customer_id
    GROUP BY
        fo.customer_id,
        fo.cohort_month,
        DATEDIFF(month, fo.cohort_month, DATEFROMPARTS(YEAR(co.order_date),
MONTH(co.order_date), 1))
),
cohort_sizes AS (
    SELECT
        cohort_month,
        COUNT(DISTINCT customer_id) AS cohort_size
    FROM first_orders
    GROUP BY cohort_month
),
retention_counts AS (
    SELECT
        cohort_month,
        month_number,
        COUNT(DISTINCT customer_id) AS retained_customers
    FROM monthly_orders
    GROUP BY cohort_month, month_number
)
SELECT
    FORMAT(rc.cohort_month, 'yyyy-MM') AS cohort,
    rc.month_number,
```

```
    cs.cohort_size,
    rc.retained_customers,
    ROUND((rc.retained_customers * 100.0 / cs.cohort_size), 2) AS retention_rate
FROM retention_counts rc
JOIN cohort_sizes cs ON rc.cohort_month = cs.cohort_month
WHERE rc.month_number >= 0;
 ;

--
_____
_____
--
_____
_____
_

-- Step 1: Identify cohort month (first purchase month for each customer)
WITH cohort AS (
    SELECT
        customer_id,
        DATEFROMPARTS(YEAR(MIN(order_date)), MONTH(MIN(order_date)), 1) AS
cohort_month
    FROM customer_orders
    GROUP BY customer_id
),

-- Step 2: Get all order months for every order
orders_by_month AS (
    SELECT
        customer_id,
        DATEFROMPARTS(YEAR(order_date), MONTH(order_date), 1) AS order_month
    FROM customer_orders
),

-- Step 3: Join to match cohort month and all order months for each customer
cohort_analysis AS (
    SELECT
        c.customer_id,
        c.cohort_month,
        o.order_month
    FROM cohort c
    JOIN orders_by_month o ON c.customer_id = o.customer_id
)
```

```
-- Step 4: Count how many customers from each cohort ordered in subsequent months
SELECT
    cohort_month,
    order_month,
    COUNT(DISTINCT customer_id) AS retained_customers
FROM cohort_analysis
GROUP BY cohort_month, order_month
ORDER BY cohort_month, order_month;
```